Imperative vs Declarative UI: jQuery and React Comparison

In this lab I understood that two different approaches to building user interface were implemented an imperative approach using jQuery and a declarative approach using React. Although both versions achieve the same goal : toggling a highlight on a paragraph they differ greatly in how the logic and UI updates are handled.

The imperative implementation with jQuery focuses on how to change the UI step by step. When the user clicks the button the code automatically selects the DOM elemt using ID and manually adds or removes the highlight class. This says that developer must explicitly tell the browser which element to find and what action to perform on it. As discussed in Chapter 1 under 'Declarative UI structures', this approach tightly couples logic with DOM manipulation, which can become harder to manage as the UI grows.

In contrast, the React implementation followa a declarative approach. Instead of directly modifiying the DOM, React uses state to describe what the UI should look like at any moment. In the example, the useState hook stores whether the paragraph should be highlighted. When the button is clicked, the state changes, and React automatically re-renders the component. This reflects the concept of "Data changes over time", where UI updates are a result of changing data, not manual DOM commands.

React's state + re-render model avoids many manual steps such as selecting elements or toggling classes directly. The developer only describes the desired UI outcome based on the current state, and React handles the rest efficiently using its virtual DOM.

This declarative approach scales much better for larger applications. As the number of elements and interactions increases, imperative code becomes complex and error-prone. React's structured components and predictable state management make complex UIs easier to maintain, debug, and extend which is why declarative UI is preferred in modern frontend development.